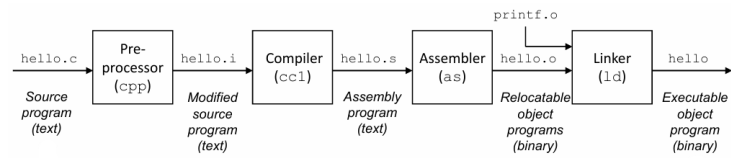
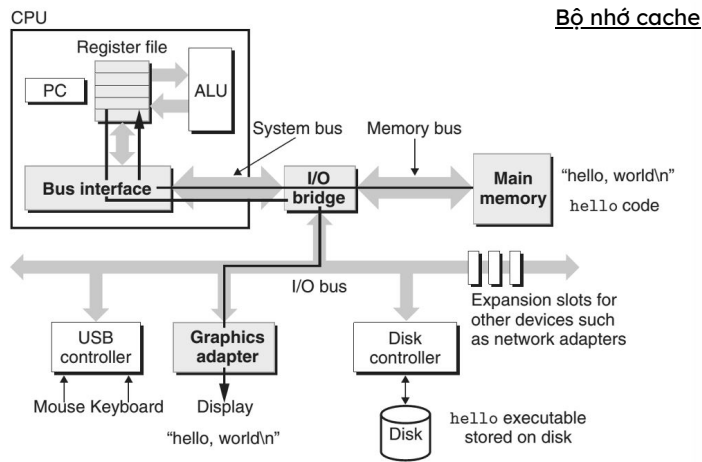
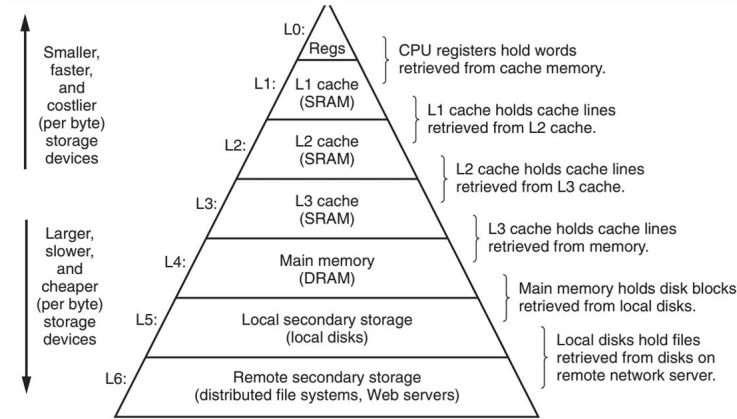


Quá trình biên dịch & liên kết



Kiến trúc phân cấp bộ nhớ



Bộ nhớ cache

Các hệ biểu diễn số

- Hệ thập phân: 10, 110, 25, 97,...
- Hệ thập lục phân: thêm tiền tố 0x, không phân biệt hoa thường (0x10, 0x25, 0xA, 0xcd...)
- Hệ nhị phân: thêm tiền tố 0b hoặc 0B trước số (0b10, 0B100101)

Bitwise operations

And (&)

- A & B = 1 khi cả A=1 và B=1

&	0	1
0	0	0
1	0	1

Or (|)

- A | B = 1 khi hoặc A=1 hoặc B=1

	0	1
0	0	1
1	1	1

Not (~)

- ~A = 1 khi A=0

~	0	1
0	1	
1	0	

Exclusive-Or (Xor) (^)

- A ^ B = 1 khi A và B khác nhau, và ngược lại

^	0	1
0	0	1
1	1	0

Các phép toán dịch bit (shift)

- Dịch trái: x << n
 - n bit bên trái bị bỏ đi
 - Điền vào bên phải n bit 0
- Dịch phải: x >> n
 - n bit bên phải bị bỏ đi dần
 - Dịch phải luận lý:
 - Không quan tâm dấu của x
 - Điền vào bên trái n bit 0
 - Dịch phải toán học
 - Quan tâm đến dấu của x
 - Điền vào bên trái n bit dấu
- Các phép dịch bit <=> phép nhân, chia
 - a << n <=> a * 2^n
 - a >> n <=> a / 2^n

Argument x	01100010
<< 3	00010000
Log. >> 2	00011000
Arith. >> 2	00011000

Argument x	10100010
<< 3	00010000
Log. >> 2	00101000
Arith. >> 2	11101000

Biểu diễn số nguyên (integer)

- Quy ước: trong hệ biểu diễn w-bit, các bit được đánh thứ tự từ 0 đến w-1 từ phải sang trái.
 - Ví dụ: Hệ 8-bit (w=8) 0 1 0 0 1 1 1 0 7 6 5 4 3 2 1 0
- Số không dấu (unsigned): Tất cả các bit đều biểu diễn giá trị
 - Tính giá trị: $B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$
 - Giá trị lớn nhất: Tất cả các bit là 1 = $2^w - 1$
 - Giá trị nhỏ nhất: Tất cả các bit là 0 = 0
- Số có dấu (signed): Bit trọng số cao nhất (w-1) biểu diễn dấu
 - 0: không âm / 1: âm
 - Tính giá trị: $B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$
 - Giá trị lớn nhất: Bit dấu 0, tất cả bit còn lại là 1 = $2^{(w-1)} - 1$
 - Giá trị nhỏ nhất: Bit dấu 1, tất cả bit còn lại là 0 = $-2^{(w-1)}$

Ví dụ: biểu diễn 8-bit có dấu 0000 0110 = 6 / 1100 0001 = -63

Biểu diễn số đối (negation)

Cho x = 15213 = 0011 1011 0110 1101; y = -15213 = bù 2 của x

- Công thức bù 2: -x = ~x + 1
- B1: Thực hiện phép ~ trên biểu diễn nhị phân của 15213
 - ~x = ~0011 1011 0110 1101 = 1100 0100 1001 0010
- B2: Cộng thêm 1 vào bit thấp nhất bên phải
 - ~x + 1 = 1100 0100 1001 0010 + 1 = 1100 0100 1001 0011

Ảnh xạ giữa số có và không dấu

Số biểu diễn với n bit, với bit trọng số cao nhất = 1:

- Giá trị không dấu (unsigned) = giá trị có dấu (signed) + 2^n
- Giá trị có dấu (signed) = giá trị không dấu (unsigned) - 2^n
- Mặc định trong C là số nguyên có dấu (signed).
- Số nguyên không dấu (unsigned): thêm hậu tố U phía sau (0U, 4294967295U)
- Lưu ý: trong biểu thức chứa cả số có dấu và không dấu, các số có dấu sẽ được chuyển sang không dấu. (<, >, ==, <=, >=)

Cho đoạn mã C

```
#include <stdio.h>

int main()
{
    int a;
    unsigned int b;

    printf("Enter an integer: ");
    scanf("%d", &a);
    printf("Enter an unsigned integer: ");
    scanf("%u", &b);

    printf("Your a: %d", a);
    printf("Your b: %u", b);
}
```

Do 1 được ép kiểu
→ biểu diễn nhị phân sẽ được
tính thành giá trị khác

Nhập a = -1 và b = -1, dự đoán và giải thích kết quả chương trình? ★

A. Chương trình lỗi

B. Chương trình in ra
2 giá trị -1

C. Chương trình in ra giá
trị -1 và 1 giá trị khác

Phép cộng

- Tổng thực tế có thể yêu cầu w+1 bit, tuy nhiên hệ biểu diễn w bit bỏ bit cao nhất (MSB). → Tràn số (overflow)

Operands: w bits

u

+ v

True Sum: w+1 bits

u + v

Discard Carry: w bits

Add_w(u, v)

Ví dụ:

- Cộng số không dấu (unsigned): 8 + 8 = 1000 + 1000 = 1 0000 = 0
- Cộng số có dấu (signed): 7 + 7 = 0111 + 0111 = 1110 = -2

Phép nhân

- Tích thực tế có thể yêu cầu 2*w bit, tuy nhiên hệ biểu diễn w bit bỏ các bit cao hơn w → Tràn số (overflow)
- Phép nhân có thể khác nhau trong một vài trường hợp của số có dấu và không dấu (Các bit thấp vẫn giống nhau)

Operands: w bits

u

* v

True Product: 2*w bits

u * v

Discard w bits: w bits

UMult_w(u, v)

Byte = 8 bits

- Hệ nhị phân: 0b00000000 đến 0b11111111
- Hệ 10 (decimal): từ 0 đến 255
- Hệ 16 (hexadecimal): 0x00 đến 0xFF

Tổ chức bộ nhớ theo byte

- Mỗi byte trong bộ nhớ được xác định = địa chỉ.
- Một địa chỉ như một index trong mảng byte đó.
- Pointer (con trỏ) dùng để chứa một địa chỉ.
- Lưu ý: hệ thống cung cấp các không gian địa chỉ riêng cho mỗi tiến trình.

Các kiểu dữ liệu (đơn vị: bytes)

Kiểu dữ liệu C	Hệ thống 32-bit	Hệ thống 64-bit	Hệ thống x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	8	8
float	4	4	4
double	8	8	8
long double	-	-	10/16
pointer	4	8	8

16	10	2
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Kích thước phụ thuộc vào kích thước của 1 địa chỉ

Tổ chức bộ nhớ theo word

- Địa chỉ xác định vị trí của byte = Địa chỉ của byte đầu trong word
- Địa chỉ của các word tiếp theo cách 4 (32 bit) hoặc 8 byte (64 bit)

Thứ tự byte - Byte ordering

- Big Endian: byte có trọng số thấp nhất nằm ở địa chỉ cao nhất
- Little Endian: byte có trọng số thấp nhất nằm ở địa chỉ thấp nhất
- Ví dụ: Cho biến x = 0x1234567 ; Địa chỉ lưu x là 0x100

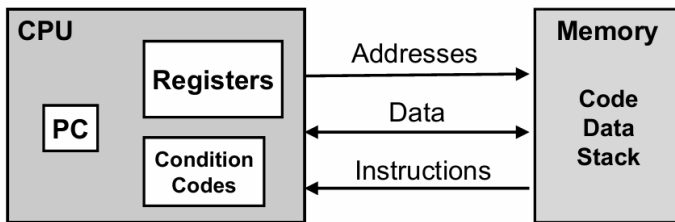
Big Endian	0x100	0x101	0x102	0x103				
		01	23	45	67			

Little Endian	0x100	0x101	0x102	0x103				
		67	45	23	01			

Biểu diễn chuỗi (strings)

- Là một mảng các ký tự, mỗi ký tự ở dạng mã ASCII (Chuẩn 7-bit)
- '0' = 0x30 → Số i tương ứng với mã 0x30 + i
- String cần được kết thúc bằng null = giá trị 0 (≠ ký tự '0').
- Thứ tự byte của hệ thống không ảnh hưởng đến cách lưu chuỗi, ký tự đầu tiên luôn luôn lưu ở địa chỉ thấp nhất

Góc nhìn của mã assembly



- PC: Program counter - Địa chỉ instruction tiếp theo cần thực thi
- Register file (thanh ghi): Dữ liệu để lưu dữ liệu chương trình
- Condition codes: Lưu thông tin trạng thái về các phép tính toán học hoặc logic được thực hiện gần nhất. Được dùng để rẽ nhánh có điều kiện.
- Bộ nhớ: Mảng các byte được đánh địa chỉ, chứa code và dữ liệu người dùng, Stack hỗ trợ các thủ tục (procedures)

Các thanh ghi IA32 - 8 thanh ghi 32 bit

Mục đích chung	%eax	%ax	%ah	%al	Kết quả
	%ecx	%cx	%ch	%cl	counter
	%edx	%dx	%dh	%dl	data
	%ebx	%bx	%bh	%bl	base
	%esi	%si			source index
	%edi	%di			destination index
	%esp	%sp			stack pointer
	%ebp	%bp			base pointer
Các thanh ghi 16-bit ảo (tương thích ngược)					

Các tổ hợp toán hạng cho movl

*Lưu ý: Không thể thực hiện chuyển dữ liệu bộ nhớ - bộ nhớ với duy nhất 1 instruction!

	Source	Dest	Src, Dest	C Analog
movl	Imm	Reg	movl \$0x4, %eax	temp = 0x4;
		Mem	movl \$-147, (%eax)	*p = -147;
	Reg	Reg	movl %eax, %edx	
		Mem	movl %eax, (%edx)	*p = temp;
	Mem	Reg	movl (%eax), %edx	temp = *p;
		Mem		

Dạng đánh địa chỉ bộ nhớ đầy đủ tổng quát nhất

D(Rb, Ri, S) ==> Mem[Reg[Rb]+S*Reg[Ri]+ D]

- D: Hằng số "dịch chuyển" 1, 2, hoặc 4 bytes
- Rb: Base register: Bất kỳ thanh ghi nào được hỗ trợ
- Ri: Index register: Bất kỳ thanh ghi nào, ngoại trừ %rsp, %esp
- S: Scale: 1, 2, 4, hoặc 8

Lưu ý: Suffix cho lệnh mov trong AT&T quyết định số byte dữ liệu

- movb = 1 byte ▪ movw = 2 bytes ▪ movl = 4 bytes
- movq = 8 bytes (dùng với các thanh ghi x86_64)
- mov = Số bytes tùy ý (phù hợp với tất cả số byte ở trên)
- Các thanh ghi dùng trong lệnh mov cần phù hợp với suffix

Instruction tính toán địa chỉ: leal Source, Dest

- Source là biểu thức tính toán địa chỉ, gán Dest thành địa chỉ được tính toán bằng biểu thức trên.
- Tác dụng: Tính toán địa chỉ ô nhớ mà không truy xuất, tính toán biểu thức toán học có dạng x + k*i + d (i = 1, 2, 4, hoặc 8)

Condition codes

- CF - Carry Flag (for unsigned) - 1 khi tràn số không dấu.
- SF - Sign Flag (for signed) - 1 khi kết quả là số âm.
- ZF - Zero Flag - 1 khi kết quả là số 0.
- OF - Overflow Flag (for signed) - 1 khi xảy ra tràn số có dấu.
- Chứa trong thanh ghi %eflag / %rflag
- Gán giá trị cho các condition codes: Gán ngầm qua các phép tính toán học hoặc Gán tường minh: các lệnh so sánh, test.
- Không được gán giá trị bằng lệnh leal!

Gán tường minh = các lệnh so sánh:

- cmpl Src2, Src1 = cmpl b, a <=> a - b mà không lưu kết quả
 - CF được gán nếu có nhớ bit ở most significant bit
 - ZF được gán nếu a == b
 - SF được gán nếu (a - b) < 0 (phép trừ có dấu - âm)
 - OF được gán nếu tràn số bù 2 (có dấu)

Gán tường minh bằng lệnh test:

- testl Src2, Src1 = testl b, a <=> a & b mà không lưu kết quả
 - ZF được gán khi a & b == 0
 - SF được gán khi a & b < 0

Rẽ nhánh có điều kiện - Các câu lệnh jump

- Các lệnh jump thường kết hợp với các lệnh so sánh/test
 - Kết quả của lệnh so sánh/test quyết định có jump hay không.
- cmpl src2, src1; jX label

jX	Điều kiện	Mô tả
jmp	1	Nhảy không điều kiện
jz	ZF	Equal / Zero
jnz	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative
jg	~(SF^OF) & ~ZF	Greater (Signed)
jge	~(SF^OF)	Greater or Equal (Signed)
jl	(SF^OF)	Less (Signed)
jle	(SF^OF) ZF	Less or Equal (Signed)
ja	~CF & ~ZF	Above (unsigned)
jb	CF	Below (unsigned)

sall	Src, Dest	Dest = Dest << Src	Cũng được gọi là shll
sarl	Src, Dest	Dest = Dest >> Src	Arithmetic (shift phải toán học)
shrl	Src, Dest	Dest = Dest >> Src	Logical (shift phải luận lý)